3

Bayesian Optimization Algorithm

The previous chapter argued that using probabilistic models with multivariate interactions is a powerful approach to solving problems of bounded difficulty. The Bayesian optimization algorithm (BOA) combines the idea of using probabilistic models to guide optimization and the methods for learning and sampling Bayesian networks. To learn an adequate decomposition of the problem, BOA builds a Bayesian network for the set of promising solutions. New candidate solutions are generated by sampling the built network.

The purpose of this chapter is threefold. First, the chapter describes the Bayesian optimization algorithm (BOA), which uses Bayesian networks to model promising solutions and bias the sampling of new candidate solutions. Second, the chapter describes how to learn and sample Bayesian networks. Third, the chapter tests BOA on a number of challenging decomposable problems.

The chapter starts by presenting the basic procedure of BOA. Section 3.2 describes Bayesian networks. Section 3.3 discusses how to learn the structure and parameters of Bayesian networks given a sample from an unknown target distribution. The section provides two approaches to measuring the quality of each candidate model: (1) Bayesian metrics and (2) minimum description length metrics. Additionally, the section describes a greedy algorithm for learning the structure of Bayesian networks. Section 3.4 describes how to sample a Bayesian network to generate new candidate solutions. Section 3.5 closes the chapter by presenting initial experiments indicating good scalability of BOA on problems of bounded difficulty.

3.1 Description of BOA

The Bayesian optimization algorithm (BOA) [129, 130, 132] evolves a population of candidate solutions by building and sampling Bayesian networks. BOA can be applied to black-box optimization problems where candidate solutions

Martin Pelikan: *Hierarchical Bayesian Optimization Algorithm*, StudFuzz **170**, 31–48 (2005)
www.springerlink.com

```
Bayesian optimization algorithm (BOA)
  t := 0;
  generate initial population P(0);
  while (not done) {
    select population of promising solutions S(t);
    build Bayesian network B(t) for S(t);
    sample B(t) to generate O(t);
    incorporate O(t) into P(t);
    t := t+1;
  };
```

Fig. 3.1. The pseudocode of the Bayesian optimization algorithm (BOA)

are represented by fixed-length strings over a finite alphabet, but for the sake of simplicity, only binary strings are considered in most of this chapter.

BOA generates the initial population of strings at random with a uniform distribution over all possible strings. The population is updated for a number of iterations (generations), each consisting of four steps. First, promising solutions are selected from the current population using a GA selection method, such as tournament or truncation selection. Second, a Bayesian network that fits the population of promising solutions is constructed. Third, new candidate solutions are generated by sampling the built Bayesian network. Fourth, the new candidate solutions are incorporated into the original population, replacing some of the old ones or all of them.

The above four steps are repeated until some termination criteria are met. For instance, the run can be terminated when the population converges to a singleton, the population contains a good enough solution, or a bound on the number of iterations has been reached. The basic BOA procedure is outlined in Fig. 3.1.

There are a number of alternative ways to perform each step. The initial population can be biased according to prior problem-specific knowledge [160, 166]. Selection can be performed using any popular selection method. Various algorithms can be used to construct the model and there are several approaches to evaluating quality of candidate models. Additionally, the measure of model quality can incorporate prior information about the problem to enhance the estimation and, as a consequence, to improve the efficiency.

The next section describes Bayesian networks and discusses techniques for learning and sampling Bayesian networks.

3.2 Bayesian Networks

A Bayesian network [88, 123] is defined by two components:

(1) **Structure.** The structure is encoded by a directed acyclic graph with the nodes corresponding to the variables in the modeled data set (in this case, to the positions in solution strings) and the edges corresponding to conditional dependencies.
(2) **Parameters.** The parameters are represented by a set of conditional probability tables specifying a conditional probability for each variable given any instance of the variables that the variable depends on.

Mathematically, a Bayesian network encodes a joint probability distribution given by

$$p(X) = \prod_{i=1}^{n} p(X_i|\Pi_i) \ , \tag{3.1}$$

where $X = (X_1, \ldots, X_n)$ is a vector of all the variables in the problem; Π_i is the set of parents of X_i in the network (the set of nodes from which there exists an edge to X_i); and $p(X_i|\Pi_i)$ is the conditional probability of X_i given its parents Π_i.

A directed edge relates the variables so that in the encoded distribution, the variable corresponding to the terminal node is conditioned on the variable corresponding to the initial node. More incoming edges into a node result in a conditional probability of the variable with the condition consisting of all parents of this variable. In addition to encoding dependencies, each Bayesian network encodes a set of independence assumptions. Independence assumptions state that each variable is independent of any of its antecedents in the ancestral ordering, given the values of the variable's parents.

A simple example Bayesian network structure is shown in Fig. 3.2. The example network encodes a number of conditional dependencies. For instance, the speed of the car depends on whether it is raining and/or radar is enforced. The road is most likely wet if it is raining. Additionally, the network encodes a number of simple and conditional independence assumptions. For instance, the radar enforcement is independent of whether it is raining or not. A more complex conditional independence assumption is that the probability of an accident is independent of whether the radar is enforced, given a particular speed and whether the road is wet. To fully specify the Bayesian network

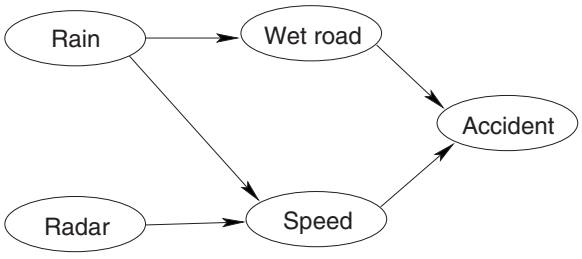


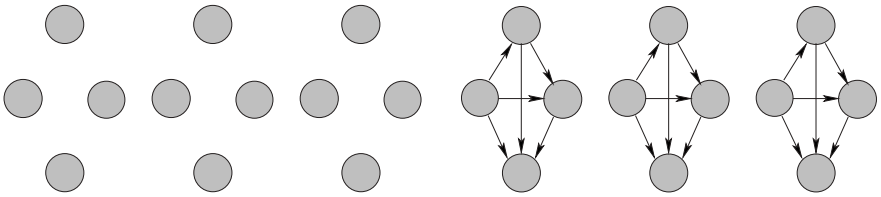
Fig. 3.2. An example Bayesian network structure

Table 3.1. If we encode the speed of a car using two values (high and low), the conditional probability table for the probability of an accident could be defined by this table. Note that the last four entries in the table are unnecessary, because they can be computed using the remaining ones (the sum of all conditional probabilities with a fixed condition is 1)

Accident	Wet Road	Speed	$p(\text{Accident}\|\text{Wet Road}, \text{Speed})$
Yes	Yes	High	0.18
Yes	Yes	Low	0.04
Yes	No	High	0.06
Yes	No	Low	0.01
No	Yes	High	0.82
No	Yes	Low	0.96
No	No	High	0.94
No	No	Low	0.99

with the structure shown in the figure, it would be necessary to add a table of conditional probabilities for each variable. An example conditional probability table is shown in Table 3.1.

It is important to understand the semantics of Bayesian networks in the framework of PMBGAs. Conditional *dependencies* will cause the involved variables to remain in the configurations seen in the selected population of promising solutions. On the other hand, conditional *independencies* lead to the mixing of bits and pieces of promising solutions in some contexts (the contexts are determined by the variables in the condition of the independency). The complexity of a proper model is directly related to a proper problem decomposition discussed in Chap. 1. If the problem was linear, a good network would be the one with no edges (see Fig. 3.3(a)); the effects of using an empty network are the same as those of using population-wise uniform crossover. On the other hand, if the problem consisted of traps of order k, the network should be composed of fully connected sets of k nodes, each corresponding to one trap, with no edges between the different groups (see Fig. 3.3(b)); the



(a) No interactions. (b) Separable blocks of order 4.

Fig. 3.3. A good model for a problem with no interactions and a problem consisting of separable subproblems of order 4

effects of using such a model would be the same as those of population-wise building-block crossover. More complex problems lead to more complex models, although many problems can be solved with quite simple networks despite the presence of nonlinear interactions of high order.

The following section discusses how to learn a Bayesian network given a data set. Subsequently, we describe the probabilistic logic sampling, which can be used to sample the distribution encoded by a Bayesian network and its parameters.

3.3 Learning Bayesian Networks

For a successful application of BOA, it is necessary that BOA is capable of *learning* a network that reflects the dependencies and independencies that decompose the problem properly. There are two subtasks of learning a Bayesian network:

(1) **Learning the structure.** First, the structure of a network must be determined. The structure defines conditional dependencies and independencies encoded by the network.
(2) **Learning the conditional probabilities.** The structure also identifies conditional probabilities that must be specified for a complete model. After learning the structure, the values of the conditional probabilities with respect to the final structure must be learned.

In BOA, learning the parameters for a given structure is simple, because the value of each variable in the population of promising solutions is specified; in other words, we can assume complete data. To maximize the likelihood of the model with a fixed structure and complete data, the probabilities should be set according to the relative frequencies observed in the modeled data (in our case, the selected set of promising solutions) [79]. Thus, the parameters can be learned by iterating through all selected solutions and computing relative frequencies of different partial solutions.

As an example of learning the parameters of a Bayesian network, recall probabilistic uniform crossover, which can be represented by a Bayesian network with no edges. The parameters of an empty network consist of the probabilities of different values for each variable. Therefore, to determine the parameters of an empty network, it is sufficient to parse the population of promising solutions and compute the observed probabilities.

Learning the structure is a much more difficult problem. It is common to split the problem of learning the structure of Bayesian networks into two components:

1. **A scoring metric.** The scoring metric measures quality of Bayesian network structures. In GA terminology, the scoring metric specifies the fitness of a structure. Usually, the scoring metric is proportional to the likelihood

of the structure or it is equal to a combination of the likelihood and a penalty that grows with model complexity. However, other measures can be used, such as statistical tests on independence.

2. **A search procedure.** The search procedure searches the space of all possible network structures to find the best network with respect to a given scoring metric. The space of network structures can be restricted according to a bound on the complexity of networks or some other prior problem-specific knowledge.

Next, the section discusses several approaches to evaluating competing network structures. Subsequently, the section describes a greedy algorithm, which can be used to learn the structure of a Bayesian network given a scoring metric.

3.3.1 Scoring Metric

There are two approaches to measuring quality of competing network structures: (1) Bayesian metrics and (2) minimum description length metrics. Bayesian metrics [27, 79] measure the quality of each structure by computing the marginal likelihood of the structure with respect to the given data and inherent uncertainties. Minimum description length metrics [149, 150, 151] are based on the assumption that model quality is proportional to the amount of compression of the data allowed by the model.

Bayesian Metrics

Measuring the quality of a network structure is difficult because there is no information about the target network structure or its parameters. Bayesian metrics [27, 79] account for these sources of uncertainty by using Bayes rule and assigning prior distributions to both network structures as well as the parameters of each structure. The quality of a structure is measured by the marginal likelihood of the structure with respect to the given data. The marginal likelihood is computed by averaging the likelihood of the models conditioned on the observed data according to a prior distribution over all possible conditional probabilities in the model:

$$p(B|D) = \frac{p(B)}{p(D)} \int_{\theta} p(\theta|B)p(D|B,\theta)\, d\theta \ , \tag{3.2}$$

where B is the evaluated Bayesian network structure (without parameters); D is the data set; and each value of θ represents one possible way of assigning conditional probabilities in the network B. Furthermore, $p(B)$ is the prior probability of the network structure B, $p(\theta|B)$ is the prior probability of parameters θ (conditional probabilities) given B, and $p(D|B,\theta)$ denotes the probability of D given the network structure B and its parameters θ. Since

the probability of data denoted in the last equation by $p(D)$ is the same for all network structures, $p(D)$ is usually omitted when evaluating the structures.

To compute the marginal likelihood, a prior probability distribution over the parameters of each structure must be given. The Bayesian-Dirichlet metric (BD) [27, 79] assumes that the conditional probabilities follow Dirichlet distribution and makes a number of additional assumptions, yielding the following score:

$$BD(B) = p(B) \prod_{i=1}^{n} \prod_{\pi_i} \frac{\Gamma(m'(\pi_i))}{\Gamma\left(m'(\pi_i) + m(\pi_i)\right)} \prod_{x_i} \frac{\Gamma\left(m'(x_i, \pi_i) + m(x_i, \pi_i)\right)}{\Gamma(m'(x_i, \pi_i))} \,, \tag{3.3}$$

where $p(B)$ is the prior probability of the network structure B; the product over x_i runs over all instances of x_i (in the binary case these are 0 and 1); the product over π_i runs over all instances of the parents Π_i of X_i (all possible combinations of values of Π_i); $m(\pi_i)$ is the number of instances with the parents Π_i set to the particular values given by π_i; and $m(x_i, \pi_i)$ is the number of instances with $X_i = x_i$ and $\Pi_i = \pi_i$. Terms $m'(\pi_i)$ and $m'(x_i, \pi_i)$ denote prior information about the statistics $m(\pi_i)$ and $m(x_i, \pi_i)$, respectively. Here we consider K2 metric [27], which uses an uninformative prior that assigns $m'(x_i, \pi_i) = 1$ and $m'(\pi_i) = \sum_{x_i} m'(x_i, \pi_i)$.

A prior distribution over network structures specified by term $p(B)$ can bias the construction toward particular structures by assigning higher prior probabilities to those preferred structures. Prior knowledge about the structure permits the assignment of higher prior probabilities to those networks similar to the structure believed to be close to the correct one [79]. The search can also be biased toward simpler models by assigning higher prior probabilities to models with fewer edges or parameters [23, 44, 134]. If there is no prior information about the network structure, the probabilities $p(B)$ are set to a constant and omitted in the construction (uniform prior).

For further details on Bayesian metrics, please refer to Cooper and Herskovits [27] and Heckerman et al. [79].

Minimum Description Length Metrics

Minimum description length metrics [149, 150, 151] are based on the assumption that model quality is somehow proportional to the amount of compression of the data allowed by the model. A model that results in the highest compression should capture most inherent regularities. There are two major approaches to the design of MDL metrics. The first is based on a two-part coding where the score is negatively proportional to the sum of the number of bits required to store (1) the model and (2) the data compressed according to the model. The second approach is based on universal code, which normalizes the conditional probability of data given a model by the sum of the probabilities of all data sequences given the model. The normalized probability of

the data is used as the basis for computing the number of bits required to compress the data.

In this work we consider one two-part MDL metric called the Bayesian information criterion (BIC) [164] used previously in the extended compact genetic algorithm (ECGA) [70] and the estimation of Bayesian networks algorithm (EBNA) [96]. In the binary case, BIC assigns the network structure a score

$$BIC(B) = \sum_{i=1}^{n} \left(-H(X_i|\Pi_i)N - 2^{|\Pi_i|} \frac{\log_2(N)}{2} \right), \tag{3.4}$$

where $H(X_i|\Pi_i)$ is the conditional entropy of X_i given its parents Π_i; n is the number of variables; and N is the population size (the size of the training data set). The conditional entropy $H(X_i|\Pi_i)$ is given by

$$H(X_i|\Pi_i) = -\sum_{x_i,\pi_i} p(x_i,\pi_i) \log_2 p(x_i|\pi_i) , \tag{3.5}$$

where $p(x_i,\pi_i)$ is the probability of instances with $X_i = x_i$ and $\Pi_i = \pi_i$; and $p(x_i|\pi_i)$ is the conditional probability of instances with $X_i = x_i$ given that $\Pi_i = \pi_i$.

$H(X_i|\Pi_i)$ denotes the average number of bits required to store a value of X_i given a value of Π_i. BIC multiplies the entropy $H(X_i|\Pi_i)$ by the population size N to reflect the number of bits required to store the entire population. The term $\log_2(N)$ denotes the number of bits required to store one parameter of the model (one probability or frequency). The number of bits required to store each parameter is divided by two because only half of the bits really matter in practice [45]. The term with the conditional entropy ensures that the more the information about the parents of a variable enables to compress the values of the variable, the higher the value of the BIC metric. The term with the $\log_2(N)$ introduces the pressure toward simpler models by decreasing the metric in proportion to the number of parameters required to fully specify the network.

For further details on the minimum description length metrics, please see Rissanen [151] and Grünwald [67].

According to our experience, Bayesian metrics tend to be too sensitive to noise in the data and often capture unnecessary dependencies. To avoid overly complex models, the space of network structures must usually be restricted by specifying a maximum order of interactions [79, 132]. On the other hand, MDL metrics favor simple models so that no unnecessary dependencies need to be considered. In fact, MDL metrics often result in overly simple models and require large populations to learn a model that captures all necessary dependencies.

3.3.2 Search Procedure

Learning the structure of a network given a scoring metric is a difficult combinatorial problem. It has been shown that finding the best network is

NP-complete for most Bayesian and non-Bayesian metrics [22]; therefore, there is no known polynomial-time algorithm for finding the best network structure with respect to most scoring metrics. However, a simple greedy algorithm [79] often performs well and has been successfully used in a number of difficult machine learning tasks.

The greedy algorithm performs an elementary graph operation that improves the quality of the current network the most until no more improvement is possible. The network structure can be initialized to the graph with no edges or the best tree graph computed using the polynomial-time maximum branching algorithm [39]. In BOA, the initial structure can be also set to the structure learned in the previous generation. In all experiments presented in this book, the network is constructed from an empty network in every generation. The following three elementary operations are commonly used:

1. **Edge addition.** An edge is added into the network to add a new dependency.
2. **Edge removal.** An existing edge is removed from the current network to remove an existing dependency, and introduce a new independence assumption or make an existing independence assumption stronger.
3. **Edge reversal.** An existing edge is reversed to change the character of the corresponding dependency. Each edge reversal can be replaced by first removing the edge and then adding the reversed edge in its place.

The search is terminated when there is no operation that can improve the score of the current metric. It is necessary to ensure that after each performed operation on the network structure, the resulting graph represents a valid Bayesian network structure. Consequently, the operations that introduce cycles in the structure must be eliminated. Additionally, it is useful to limit the number of edges that end in any node to upper-bound the complexity of the final structure. Figure 3.4 shows the pseudocode of the greedy algorithm described above. Figure 3.5 shows an example sequence of operators to learn the Bayesian network structure from Fig. 3.2.

At the beginning of this section we said that searching for the best Bayesian network is NP-complete. This leads to an interesting observation: BOA is a search algorithm that uses another search algorithm, BOA searches within a search. Why should the problem of learning the structure of Bayesian networks be simpler than the original optimization problem BOA attempts to solve? It is premature to discuss this topic in great detail, but theoretical results of Chap. 4 will show that even if the subproblems in a proper problem decomposition deceive GAs away from the optimum, the signal for constructing a Bayesian network will still lead in the right direction. The reason for this is that the search for a good model does not attempt to solve the optimization problem, but instead it learns interactions between the variables in a performance measure for the problem. Furthermore, BOA does not require the best network, it only needs a network that encodes all important interactions in a problem (or most of them). In addition to the these two arguments, an

```
Greedy algorithm for network construction
  initialize the network B (e.g., to an empty network);
  done := 0;

  repeat
    O = all simple graph operations applicable to B;
    if there exists an operation in O that improves score(B) {
      op = operation from O that improves score(B) the most;
      apply op to B;
    }
    else
      done := 1;

  until (done=1);

  return B;
```

Fig. 3.4. The pseudocode of the greedy algorithm for learning the structure of Bayesian networks

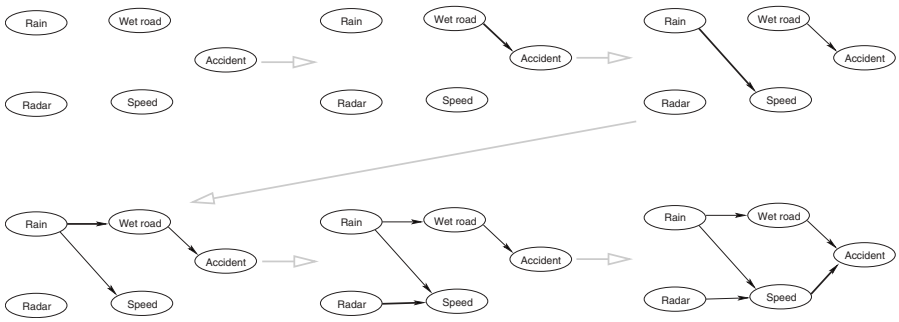


Fig. 3.5. An example sequence of steps leading to the network structure shown earlier in Fig. 3.2. For the sake of simplicity, only edge additions are used in this example

array of results presented throughout this book will support the above claim empirically.

3.4 Sampling Bayesian Networks

Once the structure and parameters of a Bayesian network have been learned, new candidate solutions are generated according to the distribution encoded by the learned network (see Equation (3.1)).

The sampling can be done using the probabilistic logic sampling of Bayesian networks [80], which proceeds in two steps. The first step computes an ancestral ordering of the nodes, where each node is preceded by its parents.

```
Algorithm for creating the ancestral ordering of the variables
  for each variable i {
    mark(i) := 0
  }
  k := 1;
  while (k <= number of variables) {
     Find variable i whose every parent x in B has mark(x)=1;
     ordered(k) := i;
     k := k+1;
     mark(i)=1;
  }
  return ordered;
```

Fig. 3.6. The pseudocode of the algorithm for computing the ancestral ordering of the variables in the Bayesian network

```
Probabilistic logic sampling of a Bayesian network B
  idx = ancestral_ordering(B);
  while (more instances needed) {
    for i=1 to length(idx) {
      generate variable idx(i) using p(idx(i)|parents(idx(i)));
    }
  }
```

Fig. 3.7. The algorithm for sampling a given Bayesian network starts by ordering the variables according to the dependencies, yielding an ancestral ordering. The variables of each new solution are generated according to the ancestral ordering using conditional probabilities encoded by the network

The basic idea is to generate the variables in such a sequence that the values of the parents of each variable are generated prior to the generation of the value of the variable itself. The pseudocode of the algorithm for computing an ancestral ordering of the variables is shown in Fig. 3.6.

In the second step, the values of all variables of a new candidate solution are generated according to the computed ordering. Since the algorithm generates the variables according to the ancestral ordering, when the algorithm attempts to generate the value of each variable, the parents of the variable must have already been generated. Given the values of the parents of a variable, the distribution of the values of the variable is given by the corresponding conditional probabilities. The second step is executed to generate each new candidate solution. A more detailed description of the algorithm for generating new candidate solutions is shown in Fig. 3.7.

3.5 Initial Experiments

This section presents the results of initial experiments using BOA on the aforementioned onemax and trap-5 problems. Additionally, BOA is tested on an additively separable deceptive function of order 3. All tested problems are of bounded difficulty – they can be decomposed into subproblems of bounded order. The performance of BOA is compared to that of the simple GA with uniform crossover and the stochastic hill climber using bit-flip mutation.

First, test problems are reviewed and discussed briefly. Next, experimental methodology is provided. Finally, the performance of BOA on the test problems is presented and compared to that of the simple GA with uniform crossover and the mutation-based hill climber.

3.5.1 Test Functions

Recall that onemax is defined as the sum of bits in the input binary string (see Equation (1.1)). The optimum of onemax is in the string of all 1s. Onemax is a unimodal function where the optimal value of each bit can be determined independently of other bits, and therefore it is easy to optimize. The purpose of testing BOA on onemax is to show that BOA can solve not only decomposable problems of bounded difficulty but also those problems that are simple and can be efficiently solved by the mutation-based hill climber.

Trap-5 is defined as the sum of 5-bit traps applied to non-overlapping 5-bit partitions of solution strings (see Sect. 1.5 on page 9 for a detailed definition). The partitioning is fixed, but there is no information about the positions in each partition revealed to BOA. Trap-5 cannot be decomposed into subproblems of order lower than 5 and therefore they can be used to analyze the scalability of BOA on nontrivial problems of bounded difficulty.

An additively separable deceptive function of order 3 [129] denoted by deceptive-3 is defined as the sum of single deceptive functions of order 3 applied to non-overlapping 3-bit partitions of solution strings. The partitioning is fixed, but there is no information about the positions in each partition revealed to BOA. The fitness contribution of each 3-bit partition is given by

$$dec_3(u) = \begin{cases} 0.9 & \text{if } u = 0 \\ 0.8 & \text{if } u = 1 \\ 0 & \text{if } u = 2 \\ 1 & \text{otherwise} \end{cases}, \tag{3.6}$$

where u is the number of ones in the input block of 3 bits. An n-bit deceptive-3 function has one global optimum in the string where all bits are equal to 1, and it has $2^{\frac{n}{3}} - 1$ local optima in strings where the bits corresponding to each deceptive partition are equal, but where the bits in at least one partition are equal to 0. Deceptive-3 represents yet another example of a nontrivial problem of bounded difficulty; however, since the order of subproblems in deceptive-3

is different compared to that in trap-5, a comparison of the performance of BOA on these two functions should reveal whether BOA performance depends on the order of subproblems in a proper problem decomposition or not.

3.5.2 Experimental Methodology

For all tested problems, 30 independent runs are performed and BOA is required to find the optimum in all the 30 runs. The performance of BOA is measured by the average number of fitness evaluations until the optimum is found. The population size is determined empirically by a bisection method so that the resulting population size is less than 10% from the minimum population size required to ensure that the algorithm converges in all the 30 runs. The bisection method used to determine the minimum population size is outlined in Fig. 3.8.

BIC is used to construct a Bayesian network in each generation, and the construction always starts with an empty network. Binary tournament selection without replacement is used in all the experiments. In most cases, better performance could be achieved by increasing selection pressure; however, the purpose of these experiments is not to show the best BOA performance, but to empirically analyze its scalability. The number of candidate solutions generated in each generation is equal to half the population size, and an elitist replacement scheme is used that replaces the worst half of the original population by offspring (newly generated solutions).

The GA with uniform crossover is also included in some of the results. All the parameters that overlap with BOA – except for population sizes – are set in the same fashion. Population sizes are also determined empirically by the bisection method. To maximize the mixing on onemax, the probability of applying crossover to each pair of parents is 1. On other problems, the crossover probability is 0.6. No mutation is used to focus on the effects of selectorecombinative search.

The performance of the mutation-based hill climber is also compared to that of BOA and the GA with uniform crossover. The mutation-based hill climber starts with a random solution. In each iteration, the hill climber applies bit-flip mutation to the current solution, and replaces the original solution by the new one if the new solution is better. The performance of the hill climber is computed according to the Markov-chain model of Mühlenbein [112], which provides the theory for computing both the optimal mutation rate as well as the expected performance.

3.5.3 BOA Performance

Figure 3.9 shows the number of fitness evaluations until BOA finds the optimum of onemax. The size of the problem ranges from $n = 100$ to $n = 500$ bits. The number of fitness evaluations can be approximated by $O(n \log n)$.

```
Bisection method for determining the minimum population size
  // determine initial bounds

  N := 100;
  success := are 30 independent runs with pop. size N successful?
  if (success) {
    while (success=1) {
      N := N/2;
      success := are 30 indep. runs with pop. size N successful?
    }
    low := N;
    high := 2*N;
  }
  else {
    while (success=0) {
      N := N*2;
      success := are 30 indep. runs with pop. size N successful?
    }
    low := N/2;
    high := N;
  }

  // use bisection to get within 10% of the minimum pop. size

  while ((high-low)/low>=0.10) {
    N = (high+low)/2;
    success := are 30 independent runs with pop. size N successful?
    if (success=0)
      low := N;
    else
      high := N;
  }

  return high;
```

Fig. 3.8. Bisection method for determining minimum population size

Therefore, the results indicate that BOA can solve onemax in a near-linear number of evaluations.

Figure 3.10 shows the number of fitness evaluations until BOA finds the optimum of trap-5 and deceptive-3 functions. The size of trap-5 functions ranges from $n = 100$ to $n = 250$ bits, whereas the size of deceptive-3 functions ranges from $n = 60$ to $n = 240$ bits. In both cases, the number of fitness evaluations can be approximated by $O(n^{1.65})$. Therefore, the results indicate that BOA can solve trap-5 and deceptive-3 in a subquadratic number of evaluations. Furthermore, except for the trivial case with no interactions between problem variables in onemax, the order of subproblems in a proper problem

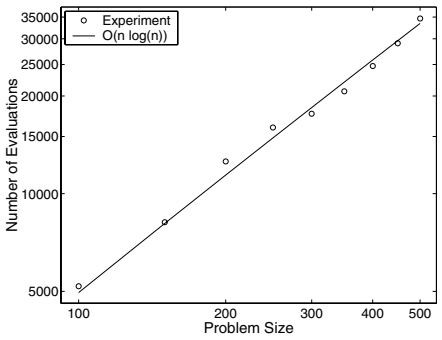
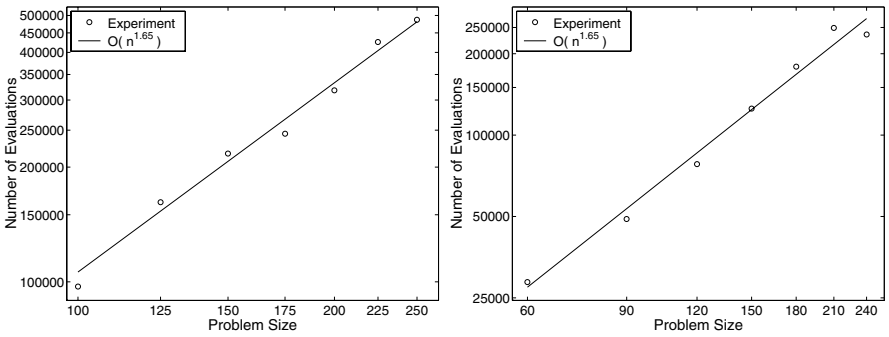


Fig. 3.9. The number of evaluations until BOA finds the optimum on onemax of varying problem size averaged over 30 independent runs. The number of evaluations can be approximated by $O(n \log n)$

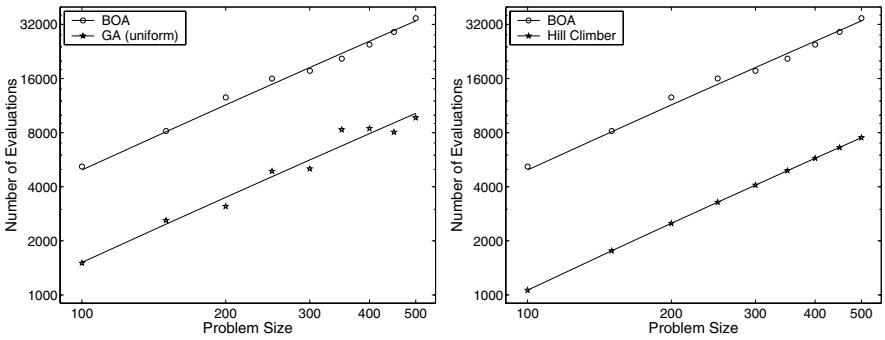


(a) Trap-5. (b) Deceptive-3.

Fig. 3.10. The number of evaluations until BOA finds the optimum of trap-5 and deceptive-3 functions. The number of evaluations can be approximated by $O(n^{1.65})$ in both cases

decomposition does not seem to affect the order of growth of the number of fitness evaluations.

Experiments on other decomposable problems indicate similar performance. In the case of onemax, the performance of BOA is close to the expected performance with population-wise uniform crossover, which is the ideal crossover to use in this case. The performance gets slightly worse for other decomposable problems where the discovery of a proper decomposition is necessary. However, in all the cases, the number of evaluations until reaching the optimum appears to grow subquadratically with the number of variables in the problem (problem size).



(a) BOA vs. the simple GA. (b) BOA vs. the hill climber.

Fig. 3.11. The comparison of BOA, the simple GA with uniform crossover, and the mutation-based hill climber on onemax

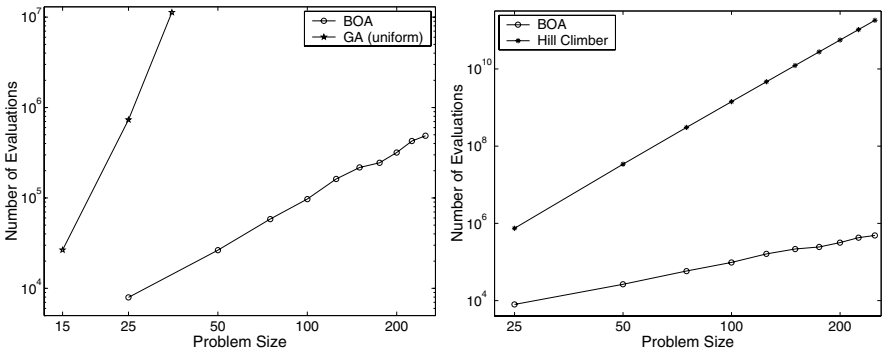
3.5.4 BOA vs. GA and Hill Climber

For onemax, both the hill climber and the simple GA with uniform crossover can be expected to converge in $O(n \log n)$ evaluations. The performance of the simple GA can be estimated by the gambler's ruin population-sizing model [70] and onemax convergence model [118]. For the simple GA with one-point or n-point crossover, the performance would get slightly worse because of a slower mixing, which results in an increased number of generations. The performance of the hill climber can be estimated using the theory of Mühlenbein [112].

Figure 3.11 compares the performance of the simple GA and the mutation-based hill climber with that of BOA for onemax. In all cases, the total number of evaluations is bounded by $O(n \log n)$; however, the performance of BOA is about 3.57-times worse than the performance of the simple GA, and the performance of the simple GA is about 1.3-times worse than the performance of the hill climber.

The reason for the worse performance of BOA is that onemax can be solved by processing each bit independently, but BOA introduces unnecessary dependencies, which lead to increased population-sizing requirements according to the gambler's ruin population-sizing model [70]. The comparison of the performances of the simple GA and the hill climber is inconclusive, because choosing a different selection pressure would change the results of the simple GA. However, it is important to note that the number of evaluations for all the algorithms grows as $O(n \ln n)$. That means that even for those simple problems that are ideal for mutation and uniform crossover, the use of sophisticated search operators of BOA does not lead to a qualitative decrease in the performance.

Figure 3.12 compares the performance of the simple GA and the mutation-based hill climber with that of BOA for trap-5. The performance of the GA and the hill climber dramatically changes compared to onemax.



(a) BOA vs. the simple GA. (b) BOA vs. the hill climber.

Fig. 3.12. The comparison of BOA, the simple GA with uniform crossover, and the mutation-based hill climber on trap-5

To solve trap-5, the simple GA requires exponentially large populations, because mixing is ineffective and requires exponentially large populations for innovative success [172]. Similar performance can be expected with other crossover methods, because the partitions of a proper problem decomposition are not located tightly in solution strings. Already for the problem of size 50, even population sizes of a half million do not result in reliable convergence; the figure only shows the results on problems of sizes 15, 20, and 25 bits. Therefore, the performance of the simple GA changes from $O(n \log n)$ for onemax to $O(a^n)$ where $a > 1$ for trap-5.

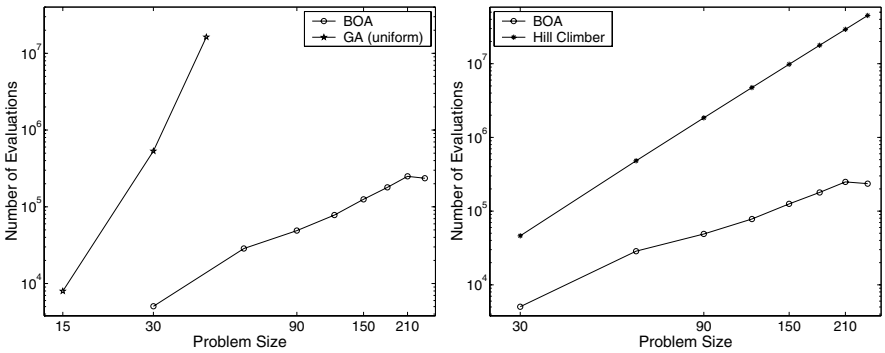
The performance of the hill climber changes from $O(n \log n)$ for onemax to $O(n^5 \ln n)$ for trap-5 [112]. In the general case, the complexity of the hill climber grows with the largest minimal order k of statistics required to solve the problem as $O(n^k \ln n)$.

Similar results can be observed for deceptive-3 (see Fig. 3.13). The number of evaluations for the simple GA grows exponentially, and the number of evaluations for the hill climber grows as $O(n^3 \log n)$.

3.5.5 Discussion

To summarize, there are three important observations:

(1) **All on onemax.** BOA, GA, and the hill climber find the optimum of onemax in approximately $O(n \log n)$ evaluations. However, BOA is outperformed by the simple GA and the hill climber within a constant factor.
(2) **GA and the hill climber on trap and deceptive functions.** The performance of the simple GA and the mutation-based hill climber significantly suffers from an increased order of an adequate problem decomposition. Both algorithms become intractable for problems of moderate difficulty.



(a) BOA vs. the simple GA. (b) BOA vs. the hill climber.

Fig. 3.13. The comparison of BOA, the simple GA with uniform crossover, and the mutation-based hill climber on deceptive-3 functions

(3) **BOA.** BOA requires a subquadratic number of evaluations until it finds the optimum of all tested problems of bounded difficulty; these results are supported by theory in the next chapter. If identifying variable interactions is unnecessary, $O(n \log n)$ evaluations until convergence can be expected. If identifying variable interactions is necessary, the performance is subquadratic. Furthermore, the performance of BOA does not depend on the location of positions corresponding to each subproblem in a proper decomposition.